

Corrigé — Examen d'entraînement LEPL1110 — Éléments Finis

Claude code

Mai 2026

Exercice 1 — Équation de Poisson et éléments finis P_2

1.1 — Formulation faible

IBP de $-\int_0^1 u''v \, dx = \int_0^1 f v \, dx$:

$$-[u'v]_0^1 + \int_0^1 u'v' \, dx = \int_0^1 f v \, dx.$$

Puisque $v \in H_0^1$: $v(0) = v(1) = 0$, les termes de bord s'annulent. On obtient $a(u, v) = \ell(v)$ $\forall v \in V$.

Essentielles vs naturelles : Les conditions de Dirichlet ($u(0) = u(1) = 0$) doivent être encodées dans $V = H_0^1$ (essentielles). Des conditions de Neumann ($u' = g$ en bord) apparaîtraient naturellement dans $\ell(v)$ via le terme de bord (naturelles).

Erreur classique : Les termes de bord s'annulent car $v = 0$ (appartenance à H_0^1), pas car $u' = 0$.

1.2 — Éléments P_2 de Lagrange

(a)

$$N_1(\xi) = \frac{\xi(\xi-1)}{2}, \quad N_2(\xi) = 1 - \xi^2, \quad N_3(\xi) = \frac{\xi(\xi+1)}{2}.$$

(b)

$$N_1' = \xi - \frac{1}{2}, \quad N_2' = -2\xi, \quad N_3' = \xi + \frac{1}{2}.$$

Partition : $N_1 + N_2 + N_3 = \frac{\xi^2 - \xi}{2} + (1 - \xi^2) + \frac{\xi^2 + \xi}{2} = 1$. ✓

1.3 — Matrice de rigidité élémentaire P_2

(a) $x(\xi) = x_e + \frac{h}{2}(1 + \xi)$, jacobien $h/2$, $d\xi/dx = 2/h$. Donc $K_{ij}^{(e)} = \frac{2}{h} \int_{-1}^1 N_i' N_j' \, d\xi$.

(b)

$$K_{11}^{(e)} = \frac{2}{h} \int_{-1}^1 \left(\xi - \frac{1}{2}\right)^2 \, d\xi = \frac{2}{h} \cdot \frac{7}{6} = \frac{7}{3h}.$$

$$K_{12}^{(e)} = \frac{2}{h} \int_{-1}^1 \left(\xi - \frac{1}{2}\right)(-2\xi) \, d\xi = \frac{2}{h} \cdot \left(-\frac{4}{3}\right) = -\frac{8}{3h}.$$

(c) Les entrées restantes : $K_{13}^{(e)} = \frac{1}{3h}$, $K_{22}^{(e)} = \frac{16}{3h}$, $K_{23}^{(e)} = -\frac{8}{3h}$, $K_{33}^{(e)} = \frac{7}{3h}$. D'où :

$$K^{(e)} = \frac{1}{3h} \begin{pmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{pmatrix}.$$

$K^{(e)}\mathbf{1}$: ligne 1 : $7 - 8 + 1 = 0$; ligne 2 : $-8 + 16 - 8 = 0$; ligne 3 : $1 - 8 + 7 = 0$. ✓

1.4 — Application : $N = 1$, $f = 2$

(a) $\xi = 2x - 1$; $N_2(x) = 4x(1 - x)$, $N_1(x) = (2x - 1)(x - 1)$, $N_3(x) = x(2x - 1)$.

$$F_0 = 2 \int_0^1 (2x^2 - 3x + 1) dx = \frac{1}{3}, \quad F_1 = 8 \int_0^1 x(1 - x) dx = \frac{4}{3}, \quad F_2 = \frac{1}{3}.$$

(b) $K = (1/3) \begin{bmatrix} 7 & -8 & 1 \\ -8 & 16 & -8 \\ 1 & -8 & 7 \end{bmatrix}$. Système réduit : $(16/3)u_1 = 4/3$, donc $u_1 = 1/4$.

(c) $u(1/2) = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. $u_1 = u(1/2)$ exactement. La solution $u = x(1 - x)$ est un polynôme de degré 2, contenu dans $V_h = P_2$, donc $u_h = u$.

Erreur classique : L'exactitude vient de $u \in V_h$, pas du fait qu'il n'y a qu'un élément.

1.5 — Lemme de Céa et convergence

(a) Si $a(\cdot, \cdot)$ est continue (M) et coercive (α), et $u_h \in V_h$ est la solution de Galerkin :

$$\|u - u_h\|_V \leq \frac{M}{\alpha} \inf_{v_h \in V_h} \|u - v_h\|_V.$$

(b) $\|u - u_h\|_{H^1} \leq Ch^k |u|_{H^{k+1}} : P_1 \rightarrow \mathcal{O}(h)$, $P_2 \rightarrow \mathcal{O}(h^2)$.

(c) $h^2 \leq 10^{-4}$, soit $h \leq 10^{-2}$, donc $N \geq 100$.

Avec P_1 il faudrait $N \geq 10\,000$ éléments pour la même précision.

Exercice 2 — Corrigé : le jeu des erreurs

① — Nombre de nœuds

```
1 x = np.linspace(0.0, 1.0, N + 1)      # ERREUR : N+1 pour P1
2 x = np.linspace(0.0, 1.0, 2*N + 1)    # CORRECT : 2N+1 pour P2
```

② — Troisième DOF

```
1 dof = [2*e, 2*e+1, 2*e+1]            # ERREUR : doublon du noeud milieu
2 dof = [2*e, 2*e+1, 2*e+2]            # CORRECT
```

③ — Jacobien

```
1 Jac = h                               # ERREUR
2 Jac = h / 2.0                         # CORRECT : jacobien de [-1,1] -> [x_e, x_e+h]
```

④ — Dérivée de N_2

```
1 dN_dxi = [xi - 0.5, 2.0*xi, xi + 0.5] # ERREUR :  $N_2' = +2xi$ 
2 dN_dxi = [xi - 0.5, -2.0*xi, xi + 0.5] # CORRECT
```

⑤ — Mauvais sens du jacobien inverse

```
1 dxi_dx = h / 2.0 # ERREUR : c'est  $dx/dxi$ , pas  $dxi/dx$ 
2 dxi_dx = 2.0 / h # CORRECT
```

⑥ — Indice J figé

```
1 J = dof[i] # ERREUR
2 J = dof[j] # CORRECT
```

⑦ — Retour incorrect

```
1 return x, K # ERREUR : retourne aussi x
2 return K # CORRECT
```

Code corrigé

```
1 import numpy as np
2
3 def assemble_stiffness_p2(N):
4     n_nodes = 2*N+1; h = 1.0/N
5     x = np.linspace(0.0, 1.0, 2*N+1) # (1)
6     K = np.zeros((n_nodes, n_nodes))
7     xi_q = [-np.sqrt(3.0/5.0), 0.0, np.sqrt(3.0/5.0)]
8     w_q = [5.0/9.0, 8.0/9.0, 5.0/9.0]
9     for e in range(N):
10         dof = [2*e, 2*e+1, 2*e+2] # (2)
11         Jac = h/2.0 # (3)
12         for q in range(3):
13             xi = xi_q[q]; w = w_q[q]
14             dN_dxi = [xi-0.5, -2.0*xi, xi+0.5] # (4)
15             dxi_dx = 2.0/h # (5)
16             dN_dx = [dN_dxi[k]*dxi_dx for k in range(3)]
17             for i in range(3):
18                 I = dof[i]
19                 for j in range(3):
20                     J = dof[j] # (6)
21                     K[I,J] += dN_dx[i]*dN_dx[j]*w*Jac
22             return K # (7)
```